TUGAS 1 — CLOUD EXPERIMENTAL ENVIRONMENT

Scale-out reaktif datang setelah burst

Laporan dan ringkasan 29 run terencana pada Online Boutique (kind, 3 worker, host 12 core), 2 November 2026. Disusun mengikuti soal: Bagian 1 environment, Bagian 2 fenomena, Bagian 3 research problem.

Ringkasan.

- **Fenomena utama: scaling delay.** Pada burst 4× (50 → 200 RPS selama 60 detik), HPA CPU baru mendapat pod tambahan 28–35 detik setelah burst mulai, dan kapasitas penuh 122–185 detik setelah burst mulai. Akibatnya 34–42% request gagal selama burst. Pre-scale terjadwal 60 detik sebelum burst menghasilkan 0 request gagal pada 3 dari 3 run.
- **Waktu scale-out terurai:** HPA membuat pod pertama 2–17 detik setelah burst mulai (deteksi cepat), tetapi pod butuh median sekitar 30 detik dari dibuat sampai Ready, dan HPA masih membuat pod sampai 64–130 detik. Yang lambat adalah rantai deteksi, penambahan bertahap, dan startup pod, bukan satu komponen saja.
- **Temuan pendukung: pod tambahan tidak otomatis ikut bekerja.** Pada 20 pod `currencyservice`, 11–12 pod hampir menganggur sementara pod lain menyentuh limit CPU. Pre-scale menghilangkan kegagalan tetapi p95 tetap di atas 300 ms pada 26–58 dari 60 detik burst. Penyebab ketimpangan ini belum terbukti.
- **Research problem:** bagaimana mendeteksi burst lebih awal dan memastikan kapasitas tambahan benar-benar melayani beban sebelum latency dan error naik.

Bagian 1 — Environment dan dataset

Komponen	Yang dijalankan
Cluster	kind `mylab99`: 1 control-plane + 3 worker, Kubernetes 1.28.13, di WSL2 (host 12 core). ingress-nginx sebagai pintu masuk.
System under test	Online Boutique, 11 service gRPC/HTTP. Limit CPU per container 125–300m (`frontend` dan `currencyservice` 200m, request 100m).
Workload generator	`loadgen.py`: profil `constant`, `burst-flash-sale`, dan `step`. Dijalankan dari mesin yang sama dengan cluster.
Monitoring	kube-prometheus-stack (Prometheus, Grafana). Collector menarik CPU, memory, network, pod Ready, dan metrik ingress untuk window run yang sama, lalu menggabungkannya ke `combined.csv` (1 baris per detik). Waktu pembuatan dan Ready pod diambil dari `kube-state-metrics`.
Dataset	Dua run baseline Tugas 1 (`constant` dan `burst-flash-sale`) ditambah rencana riset yang dijalankan otomatis oleh `plan/run_plan.py`: **29 run, 6 jam 48 menit**, 0 sel gagal. Semua ada di `results/`, ringkasannya di `results/_plan/summary.csv`.

Fase	Isi	Run
0 Baseline	`burst-flash-sale` pada 15 pod (S1), dua profil	2
1 Kapasitas	Step load 10→200→10 RPS, 60 detik per tahap, pada S1 (15 pod), S2 (36 pod), S3 (67 pod), 2 kali tiap setting	6
2 Burst statis	Burst 4× dari L0 = 50 RPS (200 RPS selama 60 detik) pada S1, S2, S3 (3 kali), plus S2 dengan pod baru di-scale (3 kali)	12
3 Autoscale	Mulai dari S1 (15 pod), target S3 (67 pod): HPA CPU, pre-scale 60 detik, pre-scale 180 detik (3 kali tiap strategi)	9

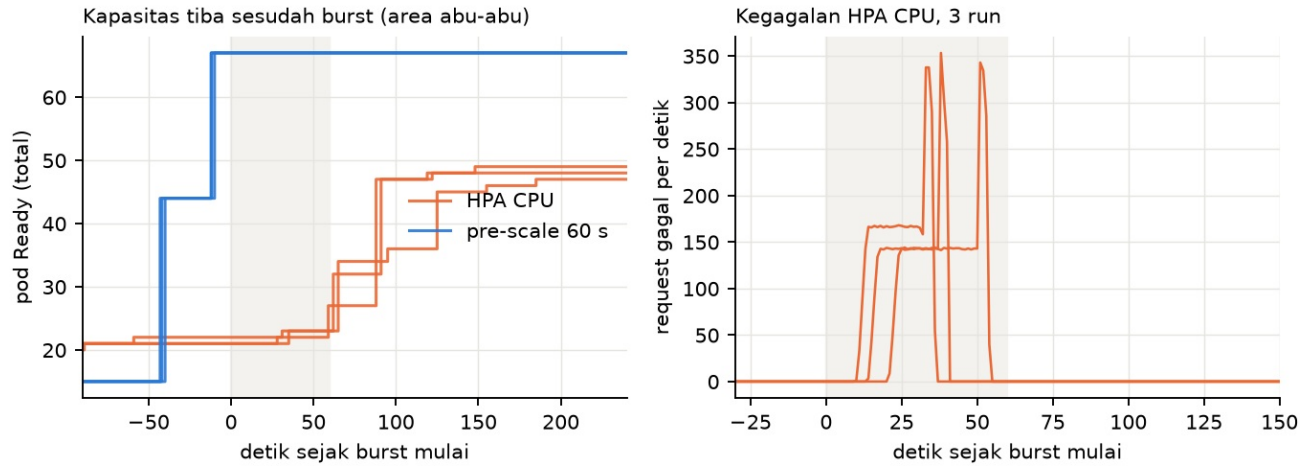
Urutan run diacak (seed tetap). Sebelum tiap run: scale ke setting target, tunggu semua pod Ready, pemanasan 120 detik pada 25 RPS. Replica tiap run tercatat di `metadata.json`. SLO yang ditetapkan sebelum run: p95 < 300 ms dan request gagal < 1%. L0 = 50 RPS adalah beban tertinggi yang masih lolos SLO pada S1 (dari Fase 1). HPA memakai target CPU 50% dari request, min = S1 dan max = S3 per service.

Yang tidak jadi dijalankan. HPA berbasis RPS (KEDA) gagal dipasang karena `helm` tidak ada di PATH saat run, sehingga strategi sinyal-RPS belum punya data. Metrik `CPU_THROTTLED` kosong di Prometheus.

Bagian 2 — Fenomena: scale-out reaktif datang setelah burst

2.1 Apa yang terjadi

Burst flash sale berlangsung 60 detik pada 4× beban normal. Pada kondisi awal S1, sistem hanya sanggup 50 RPS, sehingga burst 200 RPS memerlukan kapasitas tambahan. HPA CPU mendeteksi dan menambah pod dengan cepat, tetapi kapasitas tambahan baru melayani sesudah sebagian besar burst lewat. Selama menunggu, sekitar 145–165 dari 200 request per detik gagal.



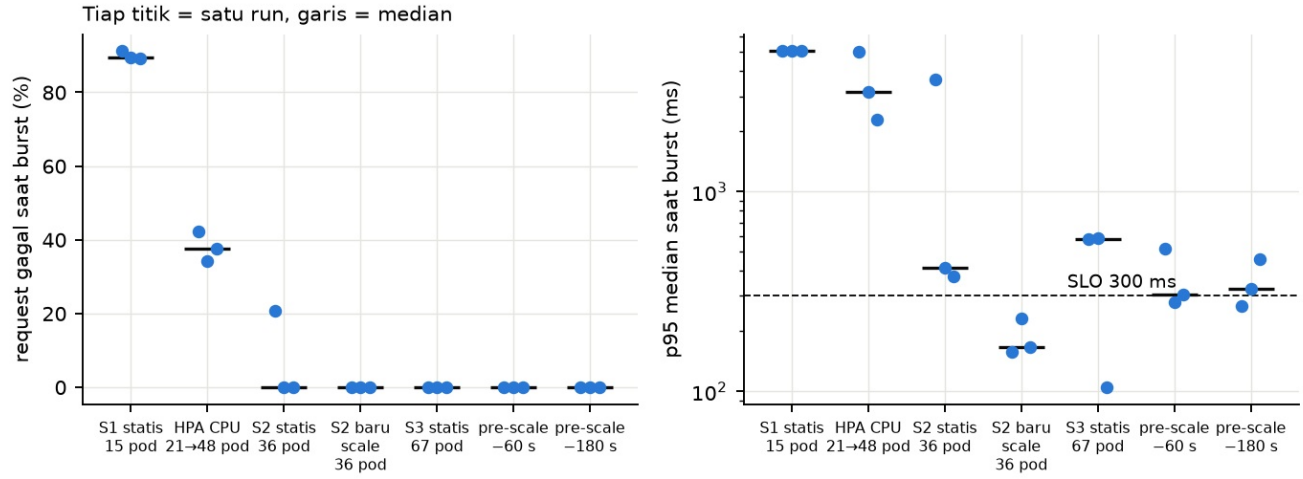
Gambar 1. Tiga run HPA CPU (oranye) dan tiga run pre-scale 60 detik (biru), waktu relatif terhadap burst mulai. Area abu-abu: burst 60 detik. Kiri: pod Ready total. Kanan: request gagal per detik pada run HPA.

Tahap scale-out (HPA CPU, 3 run)	Terukur (detik setelah burst mulai)
Pod pertama dibuat oleh HPA (deteksi)	+2 sampai +17
Kegagalan mulai	+10 sampai +21
Pod tambahan pertama Ready	+28 sampai +35
Kegagalan berhenti	+34 sampai +52 (lama 24–31 detik)
Separuh pod tambahan Ready	+65 sampai +91
Pod terakhir dibuat	+64 sampai +130
Jumlah pod puncak (47–49) Ready	+122 sampai +185
Waktu dari pod dibuat sampai Ready (semua pod baru)	median ±30 s (6–63 s)

2.2 Pada kondisi apa

- Burst 4× dari L0 (50 → 200 RPS) berdurasi 60 detik, dimulai dari 15 pod dengan HPA CPU (target 50%, maksimum 67 pod).
- Terjadi pada 3 dari 3 run HPA, dengan 4.042–5.007 request gagal per run (5,6–7,0% dari 72.000).
- Durasi burst pendek dibanding waktu scale-out. Kalau burst berlangsung lebih lama dari ±185 detik, HPA akan sempat mengejar, tetapi ini tidak diuji.

2.3 Metrik yang menunjukkan

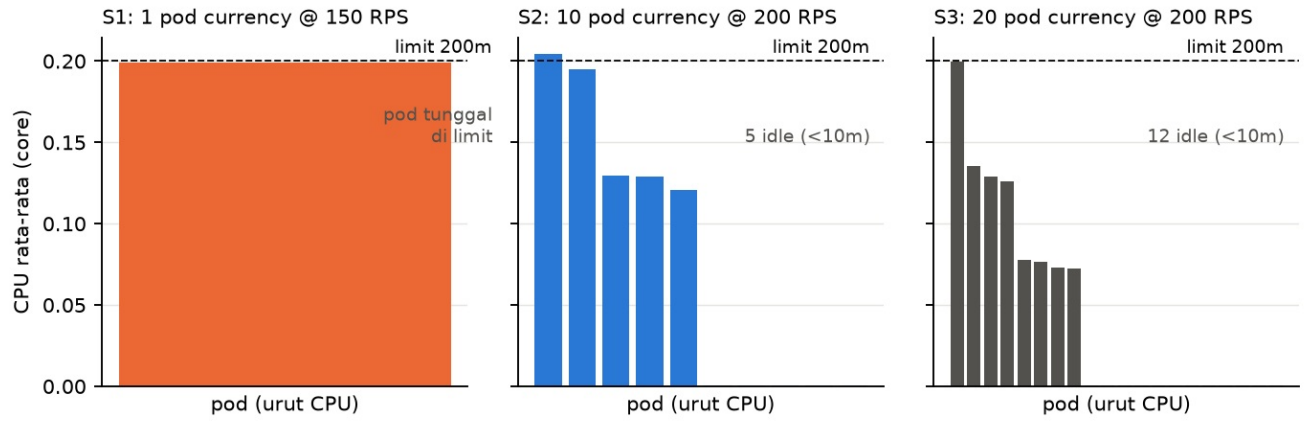


Gambar 2. Burst 4× (50 → 200 RPS, 60 detik). Satu titik per run (n = 3 tiap kolom), garis tebal median. p95 dibatasi timeout generator sekitar 5 detik.

Strategi	Pod saat burst	Gagal saat burst	p95 median saat burst (ms)	Total gagal / run
S1 statis	15	89–91%	5045	36–38 ribu
HPA CPU (target 50%)	21–23 → 47–49	34–42%	2302–5022	4,0–5,0 ribu
S2 statis	36	0%, 0%, 21%	377, 413, 3638	0, 0, 7,4 ribu
S3 statis	67	0%	105, 578, 583	0
Pre-scale 60 detik sebelum burst	67	0%	278–517	0
Pre-scale 180 detik sebelum burst	67	0%	267–458	0

Pre-scale 60 detik membuat 67 pod Ready sekitar 10–12 detik sebelum burst pada ketiga run. Request gagal nol, tetapi p95 masih di atas 300 ms pada 26, 33, dan 58 dari 60 detik burst. Jadi kapasitas yang datang tepat waktu menghilangkan kegagalan, tetapi belum memulihkan latency.

Temuan pendukung: replica tambahan tidak semuanya bekerja



Gambar 3. CPU rata-rata tiap pod `currencyservice` pada beban tertinggi di Fase 1 (S1 150 RPS, S2 dan S3 200 RPS), satu run per setting. Garis putus-putus: limit CPU 200m.

- Pada 200 RPS, S2 hanya memakai 5 dari 10 pod `currencyservice`, dan S3 hanya 8–9 dari 20. Pod yang aktif sebagian menyentuh limit 200m. Rasio pod tersibuk terhadap rata-rata: 2,6× (S2) dan 4,3× (S3).
- Kapasitas yang lolos SLO: S1 50 RPS, S2 200 RPS, S3 150 RPS. 31 pod tambahan dari S2 ke S3 tidak menambah kapasitas.
- Pada burst yang sama di S2, `currencyservice` memakai 0,40 core dengan 3–4 pod aktif (3 run), tetapi 0,63–0,66 core dengan 6 pod aktif pada S2 yang baru di-scale (3 run).

2.4 Mengapa penting di cloud

- **Elastisitas punya waktu tunda.** Cloud menjanjikan kapasitas sesuai kebutuhan, tetapi kapasitas baru tersedia 30 detik sampai 3 menit sesudah beban naik. Pada burst yang lebih pendek dari itu, autoscaling reaktif tidak melindungi request sama sekali.
- **Request yang gagal tidak bisa dipulihkan.** Ketika kapasitas tiba, burst sudah lewat. Yang tersisa adalah pengguna yang gagal checkout dan pendapatan yang hilang.
- **Sinyal CPU datang setelah antrean terbentuk.** HPA hanya bereaksi saat CPU pod sudah melewati ambang, ketika permintaan sudah menumpuk. Sinyal yang lebih awal (laju request di ingress, jadwal, atau prediksi) memberi waktu bagi pod untuk siap.
- **Jumlah pod bukan ukuran kapasitas.** Pre-scale ke 67 pod menghilangkan kegagalan, tetapi p95 tetap tinggi dan S3 tidak melampaui S2. Autoscaler yang hanya menghitung pod bisa membayar kapasitas yang tidak terpakai.
- **Ada pertukaran biaya.** Pre-scale menyiapkan 67 pod sebelum beban datang, sementara HPA hanya mencapai 47–49 pod. Kegagalan nol dibayar dengan pod yang menganggur lebih lama.

> **Batasan dan hal yang belum terbukti.**
>
> - **Mekanisme ketimpangan beban belum diverifikasi.** Dugaan: `frontend` memakai satu koneksi gRPC jangka panjang ke tiap backend lewat ClusterIP, sehingga kube-proxy memilih pod hanya saat koneksi dibuat dan pod baru tidak ikut menerima beban. Pendukung: jumlah pod aktif mendekati hitungan pemilihan acak (8 frontend ke 10 pod ≈ 5,7 pod berbeda, teramati 5; 16 frontend ke 20 pod ≈ 11,9, teramati 8–9). Koneksinya belum diperiksa dan kode `frontend` belum dibaca.
> - **Sebagian waktu startup adalah setting pod.** Readiness probe memakai periode 10 detik (`frontend` juga initial delay 10 detik), jadi waktu "dibuat sampai Ready" ikut terkuantisasi oleh setting itu. Kontribusinya belum dipisahkan dari waktu start container.
> - **S2 yang baru di-scale lebih baik dari yang dipanaskan** (p95 158–232 vs 377–413 ms), bertentangan dengan hipotesis di rencana-riset.md. Penyebabnya belum jelas.
> - **Variasi antar run besar.** S2 statis: satu dari tiga run gagal 21%. S3 statis: p95 105, 578, 583 ms. n = 3 per sel, jadi selisih kecil antar kolom tidak boleh dibaca sebagai ranking.
> - Generator dan cluster berbagi satu host, tetapi host tidak jenuh selama run (2–5 core sibuk dari 12, load average 3–6, steal dan iowait mendekati 0, dicek pada empat run). Waktu scale-out di cloud sungguhan, yang harus menyediakan node baru, akan lebih lama dari angka di sini, dan itu tidak diukur.
> - Satu-satunya pembanding sinyal adalah CPU dan jadwal tetap. Sinyal berbasis RPS belum diuji.

Bagian 3 — Research problem

3.1 Observasi yang mendasari

Eksperimen menunjukkan bahwa pada burst 60 detik, autoscaler reaktif berbasis CPU memberi kapasitas sesudah sebagian besar kerusakan terjadi: kegagalan mulai +10–21 detik, pod tambahan pertama Ready +28–35 detik, kapasitas penuh +122–185 detik. Pre-scale membuktikan bahwa kapasitas yang tersedia lebih awal menghilangkan kegagalan, tetapi tidak memulihkan p95, dan menambah pod tidak otomatis menambah kapasitas efektif karena sebagian replica tidak menerima beban.

3.2 Research question

> **Bagaimana mendeteksi burst workload lebih awal dan menyiapkan kapasitas yang benar-benar melayani beban, sehingga request gagal saat burst flash sale tetap di bawah 1% dan p95 tetap di bawah 300 ms, tanpa menyediakan seluruh kapasitas puncak sepanjang waktu?**

Termasuk kategori *autoscaling* dan *resource allocation* dalam daftar soal, dengan unsur *workload prediction* (sinyal awal) dan *resource efficiency* (biaya pod yang disiapkan lebih awal). Ia lahir dari data cluster kind yang sama, dengan harness dan metrik yang sama.

3.3 Usulan kontribusi

1. **Sinyal awal:** pemicu scale dari laju request di ingress (KEDA dengan query Prometheus, sudah disiapkan di `run_plan.py` tetapi belum terpasang) dan pre-scale terjadwal atau prediktif, menggantikan CPU.
2. **Kapasitas yang dipakai, bukan sekadar ada:** metrik kapasitas efektif (pod aktif terhadap total pod, pod tersibuk terhadap rata-rata) sebagai sinyal tambahan, dan load balancing per request (misalnya service mesh atau client-side balancing) agar replica baru ikut bekerja.
3. **Penyiapan lebih cepat dan hemat:** pod cadangan berprioritas rendah (overprovisioning) atau pool hangat, supaya kapasitas tersedia dalam hitungan detik, bukan 30 detik sampai beberapa menit.

3.4 Rancangan evaluasi

Memakai harness yang sudah ada: burst 4× dari L0 = 50 RPS, mulai dari S1, target maksimum 67 pod, 3 kali tiap sel atau lebih, urutan diacak, prosedur reset yang sama.

Pembanding	Yang diubah	Status
Baseline	HPA CPU	Sudah ada: 34–42% gagal
A	Pre-scale terjadwal	Sudah ada: 0% gagal, p95 median 278–517 ms
B	Sinyal RPS di ingress (KEDA), distribusi beban tidak diubah	Belum dijalankan
C	Sinyal RPS + load balancing gRPC per request	Belum dijalankan
D	C + pod cadangan berprioritas rendah	Belum dijalankan

Ukuran: request gagal total, p95 saat burst, waktu dari burst mulai sampai kapasitas penuh dan sampai p95 < 300 ms stabil, rasio pod aktif terhadap total pod, dan biaya (pod-detik). Kriteria berhasil: gagal < 1%, p95 < 300 ms, dan biaya pod-detik lebih rendah dari pre-scale penuh ke 67 pod.

3.5 Langkah verifikasi yang perlu dilakukan lebih dulu

- Restart semua pod `frontend` pada S2, lalu bandingkan sebaran CPU antar pod `currencyservice` sebelum dan sesudah, untuk menguji dugaan koneksi gRPC yang tertambat.
- Pasang `helm` dan KEDA, lalu jalankan sel sinyal RPS.
- Pisahkan kontribusi readiness probe terhadap waktu "dibuat sampai Ready" (jalankan dengan periode probe lebih pendek).

Lampiran — alur untuk presentasi (10–12 slide)

#	Slide	Isi dari data ini
1	Research context	Flash sale: beban naik 4× dalam sekejap, kapasitas harus siap dalam detik
2	Experimental environment	kind 1+3 node, ingress-nginx, Prometheus/Grafana, loadgen (Bagian 1)
3	Application architecture	Online Boutique, 11 service, `frontend` memanggil `currencyservice` dan service lain lewat gRPC
4	Experimental setup	Setting S1/S2/S3, SLO, urutan acak, reset, 29 run
5	Workload scenario	Step 10→200→10 RPS dan burst 50 → 200 RPS selama 60 detik
6	Metrics dan data collection	`workload.csv` + Prometheus → `combined.csv`, waktu pod dibuat dan Ready
7	Experimental results	Gambar 2 dan tabel strategi
8	Observed phenomenon	Gambar 1: kapasitas tiba setelah burst, kegagalan 24–31 detik; Gambar 3 sebagai pendukung
9	Research problem dan question	Bagian 3.2
10	Proposed contribution	Bagian 3.3
11	Evaluation design	Bagian 3.4
12	Expected contribution	Gagal < 1% dan p95 < 300 ms saat burst dengan biaya pod lebih rendah dari pre-scale penuh